Adım 0.2 — CubeIDE'de Boş F407 Projesi

Konu anlatımı ve soru-cevap · Güvenli Telemetri Düğümü (STM32F407 + FreeRTOS + lwIP + AEAD)

1. Bu adımda ne yaptık?

- CubeIDE'de **STM32CubeIDE Empty Project** sihirbazı ile, Board Selector'dan **STM32F407G-DISC1** kartını seçerek **telemetri-dugumu** projesini oluşturduk (dil: C, tür: Executable).
- Proje `firmware/` klasörünün altına, workspace de aynı klasöre açıldı.
- **Build Project** ile derledik. Sonuç: **0 errors, 1 warning**.

2. Kavram: CubeIDE ne üretir?

Empty projede CubeIDE bize hazır bir **iskelet** verir. Ağaçtaki dosyalar ve görevleri:

Dosya / klasör	Görevi
Startup/startup_stm32f407vgtx.s	Reset sonrası ilk çalışan kod: vektör tablosu, Reset_Handler. main()'e kadar olan hazırlık (0.3'te okunacak).
STM32F407VGTX_FLASH.ld	Linker script: kodu Flash'a, değişkenleri RAM'e yerleştiren harita (normal kullanım).
STM32F407VGTX_RAM.ld	Aynı harita, ama her şeyi RAM'e koyan alternatif sürüm.
Src/main.c	Boş main(): içinde sadece for(;;) sonsuz döngüsü var.
Src/syscalls.c, sysmem.c	newlib C kütüphanesinin ihtiyaç duyduğu sistem çağrıları ve bellek (malloc/heap) desteği.
Inc/	Başlık dosyaları klasörü (şimdilik boş).

3. Kavram: HAL nedir, neden almıyoruz?

HAL (Hardware Abstraction Layer), ST'nin hazır sürücü kütüphanesidir. `HAL_GPIO_WritePin()` gibi fonksiyonlar sunar; arkada register'lara yazar. Biz HAL kullanmıyoruz çünkü amaç register'ları kendimiz yazarak donanımın gerçekte nasıl çalıştığını öğrenmek ve mülakatta "HAL arkada ne yapıyor?" sorusuna cevap verebilmek.

4. Kavram: Empty ile STM32Cube proje türü

- İkisinde de **kart/MCU seçilir**; fark seçimden sonra üretilen koddadır.
- **STM32Cube** türü: `.ioc` dosyası (CubeMX arayüzü), `Drivers/` klasörü (HAL + CMSIS), `SystemClock_Config()`, `MX_GPIO_Init()` gibi otomatik üretilmiş kod gelir.
- **Empty** türü: Yukarıdaki iskelet vardır; **HAL, Drivers/ klasörü, .ioc ve otomatik üretilmiş init kodu yoktur**. Proje tamamen boş değil ama "çalışan hiçbir şey yapmayan" bir iskelettir.
- Bu yüzden başlık dosyası `stm32f4xx.h` (register adları) da şimdilik yok; gerekince CMSIS başlıklarını birlikte ekleyeceğiz.

5. Kavram: .elf dosyası nedir?

ELF = Executable and Linkable Format. Derleyici + linker'ın ürettiği **çıktı dosyasının biçimidir** ("başka sınıflarla bağlantı kuran" bir şey değildir; bağlantı işini **linker** yapar, ELF o işin sonucudur).

- İçinde **makine kodu** (.text), **başlangıç değerli değişkenler** (.data), **sıfırlanacak değişkenler** (.bss) bölümleri bulunur.
- Ayrıca **sembol tablosu** (fonksiyon/değişken adları) ve **debug bilgisi** (hangi makine kodu hangi C satırı) taşır.
- Kartın üzerinde "olduğu gibi" çalışmaz: debugger/programlayıcı (ST-LINK) ELF'ten yalnızca Flash'a gidecek kısımları (kod + .data'nın başlangıç değerleri) kartın Flash'ına yazar. Sembol ve debug bilgisi **PC'de kalır**; debug'da değişken adlarını görmemizi sağlar.

Zincir: `.c → (derleyici) → .o → (linker + .ld) → .elf → (ST-LINK) → kartın Flash'ı`

6. Kodlar ve çıktılar

Bu adımda kendi yazdığımız kod yok. CubeIDE'nin ürettiği `main.c`:

```
int main(void)
{
    /* Loop forever */
    for(;;);
}
```

Derleyici uyarısı (`main.c`, 22. satır):

```
#warning "FPU is not initialized, but the project is compiling for an FPU."
```

Anlamı: Cortex-M4'ün kayan nokta birimi (FPU) varsayılan olarak kapalıdır; proje ise FPU'lu komutlar üretecek şekilde derleniyor. FPU açılmadan float işlemi yapılırsa işlemci hata verir (fault). Bu bir hata değil, hatırlatmadır; saat ayarlarıyla birlikte kendi kodumuzla çözeceğiz (ilgili register için RM0090 / Cortex-M4 belgelerine bakacağız).

Build çıktısı (gerçek):

```
   text    data     bss     dec     hex filename
    652       0    1568    2220     8ac telemetri-dugumu.elf
```

- **text (652 bayt):** kod (ve sabitler); Flash'ta durur.
- **data (0):** başlangıç değeri olan global değişken henüz yok.
- **bss (1568):** RAM'de ayrılan, açılışta sıfırlanan alan. Bu sayının nereden geldiğini (heap/stack ayrımı dahil) 0.3'te linker script'e bakarak bulacağız; şimdilik tahmin yürütmek yeterli.

7. Soru - cevap

Soru 1: Project Type olarak Empty seçtik, STM32Cube seçmedik. Fark nedir? Projede hangi dosyalar eksik?

Doğru cevap: İkisinde de kart seçilir; fark üretilen koddadır. STM32Cube türü HAL sürücülerini (`Drivers/`), `.ioc` dosyasını ve `MX_..._Init()` gibi otomatik kodu ekler. Empty türünde bunlar yoktur; sadece startup dosyası, linker script, syscalls/sysmem ve boş bir main() gelir. Yani proje tamamen boş değil, ama donanımı yapılandıran hiçbir kod içermez.

Soru 2: .elf dosyası nedir, doğrudan kartın üzerinde mi çalışır?

Doğru cevap: ELF (Executable and Linkable Format), linker'ın ürettiği çıktı dosyasıdır; makine kodu + bölümler + sembol tablosu + debug bilgisi içerir. Kartta olduğu gibi çalışmaz: ST-LINK, içinden Flash'a gidecek kısımları kartın Flash'ına yazar; debug bilgisi PC'de kalır. "Diğer sınıflarla bağlantı" işini ELF değil, onu üreten **linker** yapar.

8. Kendini test et

- Empty proje ile STM32Cube projesi arasındaki üç somut farkı say.
- `.c → .elf` zincirinde derleyicinin ve linker'ın görevini ayır.
- FPU uyarısını kendi cümlenle anlat: neden çıkıyor, görmezden gelirsek ne olur?
- `text`, `data`, `bss` hangisi Flash'ta, hangisi RAM'de durur? (0.3'te kesinleşecek.)